Document Type: Technical Specification & Architecture Guide
Product: Text-to-SQL Analytics Platform (Enterprise Edition)
Target Audience: System Architects, Security Officers, CTOs
Date: October 2023

Enterprise Text-to-SQL Analytics Platform

1. Executive Summary

The Text-to-SQL Analytics Platform is an enterprise-grade solution designed to democratize data access by allowing business users to query complex databases using natural language. Unlike simple wrapper tools, this platform utilizes a sophisticated **multi-agent orchestration architecture** to ensure high accuracy, security, and context-awareness.

Value Proposition:

- **Zero-Code Analysis:** Business users can generate SQL, charts, and insights without technical knowledge.
- **Enterprise Security:** Built-in read-only enforcement, PII masking, and audit logging.
- **High Accuracy:** Uses a 9-step agentic pipeline to plan, validate, and self-correct queries, minimizing hallucinations.
- **Database Agnostic:** Connects to any standard SQL database (Postgres, Oracle, SQL Server, Snowflake, etc.).

2. Product Architecture

The system is built on a modular, microservices-ready architecture designed for scalability and extensibility. It decouples the user interface, API layer, agentic logic, and database connectivity.

High-Level Components

- **Frontend (UI):** A lightweight, detached interface (Streamlit for demo/internal, React-ready) that communicates strictly via REST APIs.
- **API Layer (Backend):** Built with **FastAPI** (Python 3.10+), providing asynchronous endpoints, schema validation via Pydantic v2, and secure session handling.

- **Orchestration Layer (Agno AGI):** The core "brain" of the system. It manages the lifecycle of a query, routing tasks between specialized agents (Planner, Generator, Validator).
- **LLM Gateway:** A model-agnostic abstraction layer that connects to OpenAI, Azure OpenAI, or Anthropic, handling prompt versioning and token management.
- **Data Access Layer (SQLAlchemy):** Handles secure, pooled connections to customer databases using reflection for schema discovery.

Technology Stack

Component	Technology
Language	Python 3.10+
Web Framework	FastAPI (Async)
Agent Framework	Agno AGI (Orchestration)
ORM / DB Driver	SQLAlchemy Core + Async Drivers (aiosqlite, asyncpg, etc.)
Validation	Pydantic v2, SQLParse
Security	Fernet Encryption, OAuth2 Scaffolding

3. How It Works: UI to Backend Flow

Flow 1: User Clicks "Connect"

1. **UI Input:** User provides DB credentials (host, port, user, password, db name).
2. **API Request:** `POST /connect-db` is called with payload.
3. **Validation:** Backend validates input format (Pydantic).
4. **Connectivity Test:** The **Schema Agent** attempts a lightweight connection ("ping") to the target DB.
5. **Encryption:** If successful, credentials are encrypted using a master key (Fernet symmetric encryption).
6. **Storage:** Encrypted config is stored locally or in Vault (configurable), returning a connection ID.

Flow 2: User Clicks "Introspect"

1. **Trigger:** User requests schema analysis.
2. **Reflection: Schema Agent** connects and uses SQLAlchemy reflection to read metadata (Table names, Columns, Types, PK/FKs).

3. **Semantic Analysis:** The agent samples 3-5 rows (safe mode) to detect categorical values, time columns, and potential PII.
4. **Caching:** A "semantic schema map" is generated and cached. This map is what the LLM sees, not the raw DDL, saving tokens and improving context.

Flow 3: User Clicks "Run/Ask" (The Agent Pipeline)

When a natural language question is submitted, the **Orchestrator** spins up a dedicated workflow:

1. **Intent Extraction:** The **Intent Agent** converts "Show me sales" into structured JSON (Metrics: Sales, Dimension: None, Filters: None).
2. **Planning:** The **SQL Planning Agent** looks at the cached schema and determines the join path (e.g., Join Orders to Customers on CustomerID).
3. **Generation:** The **SQL Generation Agent** writes the specific dialect SQL (e.g., PL/SQL for Oracle, T-SQL for Azure).
4. **Safety Check:** The **Validation Agent** parses the SQL. It checks for:
   - Syntax errors.
   - Destructive keywords (DROP, DELETE, TRUNCATE).
   - PII columns (checks against Deny List).
   - Row limits (enforces `LIMIT 1000`).
5. **Execution:** The **Execution Agent** runs the query with a strict timeout. If it fails, it feeds the error back to the Generator for a retry.
6. **Insight & Viz:** Results are passed to the **Insight Agent** (for text summary) and **Viz Agent** (for Plotly charts).

4. Multi-Agent Architecture

We do not rely on a single large prompt. Instead, 9 specialized agents work in concert.

Agent Name	Role & Responsibilities
1. Schema Intelligence	Connects to DB, builds semantic map, identifies fact vs. dimension tables, detects foreign keys.
2. Business Intent	Translates vague user queries into structured "Business Intent" objects (Metrics, Filters, Timeframes).
3. SQL Planner	Abstract logic planner. Decides *how* to solve the problem (e.g., "Need a CTE for monthly aggregation") without writing code.

Agent Name	Role & Responsibilities
4. SQL Generator	The coder. Takes the Plan + Schema and writes precise SQL. Handles dialect nuances (e.g., `DATE_TRUNC` vs `FORMAT`).
5. SQL Validator	The gatekeeper. Uses AST parsing to block dangerous queries. Injects `LIMIT` clauses if missing.
6. Execution Agent	Runs the query via async drivers. Captures execution time and row counts. Handles retries on connection timeouts.
7. Insight Agent	Data analyst. Reads the returned rows and provides a narrative summary, highlighting trends or outliers.
8. Viz Agent	Decides if a chart is needed. Generates a JSON spec (Plotly) for the frontend to render.
9. Recommendation Agent	Suggests 3 follow-up questions based on the current context (e.g., "Why did sales drop in Q3?").

5. Database Support & Integration

The platform uses SQLAlchemy Core, making it compatible with most standard SQL databases.

Supported Databases

- **Open Source:** PostgreSQL, MySQL, MariaDB, SQLite.
- **Enterprise:** Oracle Database, Microsoft SQL Server, IBM Db2.
- **Cloud Data Warehouses:** Snowflake, Google BigQuery, Amazon Redshift, Azure Synapse.
- **Cloud SQL:** AWS RDS, Azure SQL, Google Cloud SQL.

Integration Process

To integrate a new database, the client must provide a standard SQLAlchemy connection string. Drivers must be installed in the Python environment.

```
# Example Connection Strings
PostgreSQL: postgresql+asyncpg://user:pass@host:5432/dbname
Oracle:     oracle+cx_oracle://user:pass@host:1521/?service_name=orcl
SQL Server: mssql+pyodbc://user:pass@host/dbname?driver=ODBC+Driver+17
```

6. Scalability & Complex Schemas

Large Schema Strategy

Standard LLMs cannot ingest schemas with 10,000 tables. We handle this via:

1. **Schema Embeddings:** We vectorize table and column descriptions/names.
2. **RAG for Tables:** When a user asks a question, we first search the vector index to retrieve the top 10-20 most relevant tables.
3. **Context Window Management:** Only relevant schema fragments are injected into the SQL Generation prompt.

Benchmarks

- **Tables Supported:** Tested up to 500 tables without RAG; unlimited with RAG enabled.
- **Concurrency:** FastAPI backend supports async request handling. Horizontal scaling is achieved by deploying multiple API replicas behind a load balancer.

7. Security & Compliance

> **Security First Design**
>
> The system is designed with a "zero-trust" approach to database interaction. We never store database data, only configuration metadata.

Data Protection

- **Encryption at Rest:** Database credentials are encrypted using AES-128 (Fernet) before storage.
- **Encryption in Transit:** All API traffic is TLS 1.2+ encrypted.
- **Read-Only Enforcement:** The application should be connected via a specific DB user granted `SELECT` permissions only. The SQL Validator agent provides a secondary software-level block against DML (INSERT/UPDATE/DELETE).

PII Protection

- **Heuristic Detection:** Schema Agent flags columns like `email`, `ssn`, `phone` automatically.
- **Allow/Deny Lists:** Administrators can configure explicit deny lists. If a query requests a denied column, the Validation Agent rejects it before execution.
- **Presidio Integration:** Architecture supports integration with Microsoft Presidio for advanced PII scanning of result sets.

Audit Logging

Every interaction is logged with: Timestamp, User ID, Natural Language Query, Generated SQL, Execution Time, and Row Count.

8. SQL Correctness & Validation

Ensuring valid and safe SQL is our primary technical challenge. We solve it via a 3-layer defense:

1. **Layer 1: Syntax & Rules (Software)**
   Uses `sqlparse` library to tokenize the query. Blocks keywords (`DROP`, `GRANT`, `SHUTDOWN`). Enforces `LIMIT` clause to preventing fetching 1M+ rows.
2. **Layer 2: Self-Correction (Agentic)**
   If the database returns a SQL Error (e.g., "Column not found"), the Execution Agent captures the error message and passes it back to the Generator Agent. The Generator attempts to fix the query (up to 3 times).
3. **Layer 3: Semantic Validation (Logic)**
   The Insight Agent sanity checks the data. If the answer is "Zero Sales" but the question implies active business, it flags a potential logic error in the join.

9. LLM Provider Support

The platform is **Model Agnostic**. We provide an interface `LLMProvider` that can be implemented for any backend.

Supported Providers

- **OpenAI:** GPT-4o, GPT-4 Turbo (Recommended for highest SQL accuracy).
- **Azure OpenAI:** Fully supported for enterprise data privacy requirements.
- **Anthropic:** Claude 3.5 Sonnet (Excellent for complex logic/reasoning).
- **Local/Open Source:** Llama 3 via Ollama/vLLM (Supported but requires high-end hardware for good SQL generation).

Cost Optimization

We use structured JSON schemas to force models to be concise, reducing output token costs. We also cache schema context so it isn't regenerated for every query.

10. API Reference

All endpoints are prefixed with `/api/v1`. Authentication via Header: `X-API-Key: your-key`.

Endpoints

POST /connect-db

Register a new database connection.

```
{
  "name": "Production Sales DB",
  "type": "postgres",
  "host": "db.internal.corp",
  "port": 5432,
  "user": "analytics_ro",
  "password": "secure_password",
  "dbname": "sales_dw"
}
```

POST /introspect

Trigger schema analysis for a connection.

```
{ "connection_id": "conn_12345" }
```

POST /ask

Submit a natural language question.

```
{
  "connection_id": "conn_12345",
  "question": "What was the total revenue by region last month?"
}
```

Response:

```
{
  "sql": "SELECT region, SUM(amount) ...",
  "result_table": [...],
  "chart_config": { "type": "bar", "data": ... },
  "answer": "The total revenue was..."
}
```

11. Deployment & Operations

Installation

The application is containerized. Deployment is standard via Docker/Kubernetes.

```
docker build -t text2sql-platform .
docker run -p 8000:8000 --env-file .env text2sql-platform
```

Environment Configuration

Key environment variables:

- `TEXT2SQL_APP_MASTER_KEY`: Key for encrypting DB configs.
- `OPENAI_API_KEY` / `AZURE_OPENAI_KEY`: LLM credentials.
- `LOG_LEVEL`: INFO or DEBUG.

12. FAQ & Troubleshooting

Can it handle my specific database?

If SQLAlchemy supports it, we support it. This covers 95% of the market. For legacy mainframes, we recommend an ODBC bridge.

How do I ensure data doesn't leave my network?

Deploy the platform **on-premise** or in your **VPC**. Use Azure OpenAI or a local LLM so data never crosses public internet boundaries.

What if the SQL is wrong?

The system is probabilistic. However, with our "Validation Agent" and "Retry Loops," accuracy is significantly higher than raw ChatGPT. Users can always view and manually edit the generated SQL in the UI.

What's the cost per query?

Depends on the LLM. With GPT-4o, an average complex query (Schema + Plan + SQL + Insight) costs approximately $0.03 - $0.06.

13. Client Talking Points & Competitive Advantages

Feature	This Platform	Standard Chatbots
Schema Awareness	Deep semantic introspection & caching.	Paste DDL manually (limited).
Security	Read-only enforcement, PII masking, local execution.	Often sends data to public cloud indiscriminately.
Accuracy	Multi-agent validation & retry loops.	"One-shot" generation (often hallucinates tables).
Visualization	Auto-generates Plotly charts.	Usually text/code only.

Why Buy?

- **Reduce Data Team Backlog:** Offload 40-60% of ad-hoc reporting requests to self-service.
- **Audit Ready:** Full transparency on who asked what and what data was accessed.
- **Vendor Agnostic:** Switch from OpenAI to Anthropic to Llama without changing application code.